

Sprawozdanie z laboratorium nr 2

Obliczenia Naukowe

Jędrzej Sajnog, indeks: 279701

7 listopada 2025

Zadanie 1

Krótki opis problemu

Zadanie polega na powtórzeniu zadania 5 z poprzedniej listy laboratoryjnej, które dotyczyło obliczania sumy iloczynów skalarnych $\sum_{i=1}^5 x_i y_i$ różnymi metodami. W obecnej wersji zadania wprowadzono niewielkie modyfikacje w danych wejściowych – usunięto ostatnią cyfrę z czwartego i piątego elementu wektora x . Celem jest zbadanie, jak te drobne zmiany wpłyną na wyniki i błędy względne obliczeń.

Opis rozwiązania

Obliczamy na 4 różne sposoby:

1. **Sumowanie w przód:** standardowa kolejność od $i = 1$ do 5.
2. **Sumowanie w tył:** odwrócona kolejność od $i = 5$ do 1.
3. **Od największego do najmniejszego:** iloczyny są sortowane malejąco, a następnie sumowane.
4. **Od najmniejszego do największego:** iloczyny są sortowane rosnąco, a następnie sumowane.

Obliczenia przeprowadzono dla dwóch typów danych: ‘Float32’ (pojedyncza precyzja) oraz ‘Float64’ (podwójna precyzja). Dla każdego wyniku obliczono błąd względny δ w stosunku do dokładnej wartości sumy wynoszącej $-1.00657107 \cdot 10^{-11}$.

Wyniki i interpretacja

Wyniki dla danych oryginalnych (z listy 1)

```
=====Current type: Float32=====
True sum: -1.0065710699999998e-11
Forward: -0.4999443, : 4.966805766128285e10
Backward: -0.4543457, : 4.5137965580009605e10
Large to small: -0.5, : 4.967359135306108e10
Small to large: -0.5, : 4.967359135306108e10
```

```
=====Current type: Float64=====
True sum: -1.0065710699999998e-11
Forward: 1.0251881368296672e-10, : 11.184955313981629
Backward: -1.5643308870494366e-10, : 14.541186645165917
Large to small: 0.0, : 1.0
Small to large: 0.0, : 1.0
```

Błędy wynikają z odejmowania bliskich sobie dużych liczb, tak jak już to omawialiśmy w ramach listy 1.

Wyniki dla danych zmodyfikowanych

```
=====Current type: Float32=====
True sum: -1.0065710699999998e-11
Forward: -0.4999443, : 4.966805766128285e10
Backward: -0.4543457, : 4.5137965580009605e10
Large to small: -0.5, : 4.967359135306108e10
Small to large: -0.25, : 2.483679567603054e10

=====Current type: Float64=====
True sum: -1.0065710699999998e-11
Forward: -0.004296342739891585, : 4.268295461567235e8
Backward: -0.004296342998713953, : 4.268295718699966e8
Large to small: -0.004296342842280865, : 4.2682955632881e8
Small to large: -0.004296342842280865, : 4.2682955632881e8
```

Po drobnej modyfikacji danych (usunięcie jednej cyfry z dwóch liczb), wyniki dla 'Float32' pozostały bardzo niedokładne. Dodatkowo znacznie pogorszyły się wyniki w 'Float64'. Błąd względny wzrósł z rzędu 10^1 do rzędu 10^8 . Oznacza to, że wynik zmienił się o osiem rzędów wielkości pod wpływem zmiany danych wejściowych na 9. i 10. miejscu po przecinku.

Wnioski

Problem obliczania tej konkretnej sumy iloczynów jest **źle uwarunkowany**. Niewielka zmiana w danych wejściowych prowadzi do znacznej, nieproporcjonalnie dużej zmiany wyniku.

Zadanie 2

Krótki opis problemu

Zadanie polega na analizie funkcji $f(x) = e^x \ln(1 + e^{-x})$. Należało narysować jej wykres przy użyciu dwóch różnych programów, a następnie obliczyć jej granicę przy $x \rightarrow \infty$.

Opis rozwiązania

Wykres funkcji został wygenerowany w języku Julia (z użyciem pakietu ‘Plots’) oraz w języku Python (z użyciem ‘Matplotlib’). Następnie obliczono analitycznie granicę:

$$\lim_{x \rightarrow \infty} e^x \ln(1 + e^{-x})$$

Stosując rozwinięcie Taylora dla $\ln(1 + u)$ wokół $u = 0$, gdzie $u = e^{-x} \rightarrow 0$, otrzymujemy:

$$\ln(1 + u) = u - \frac{u^2}{2} + \dots$$

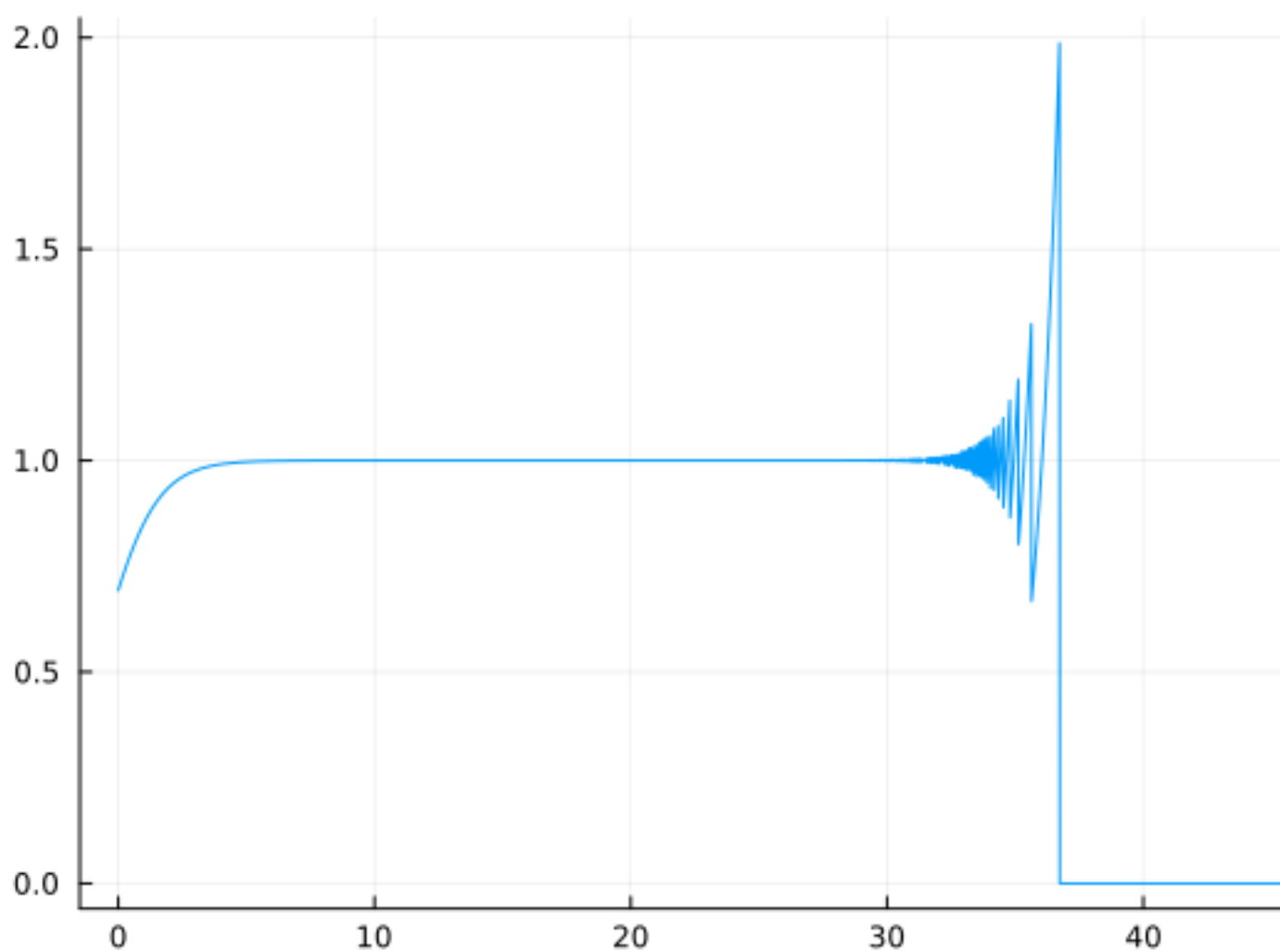
$$\ln(1 + e^{-x}) = e^{-x} - \frac{e^{-2x}}{2} + \dots$$

Podstawiając do granicy:

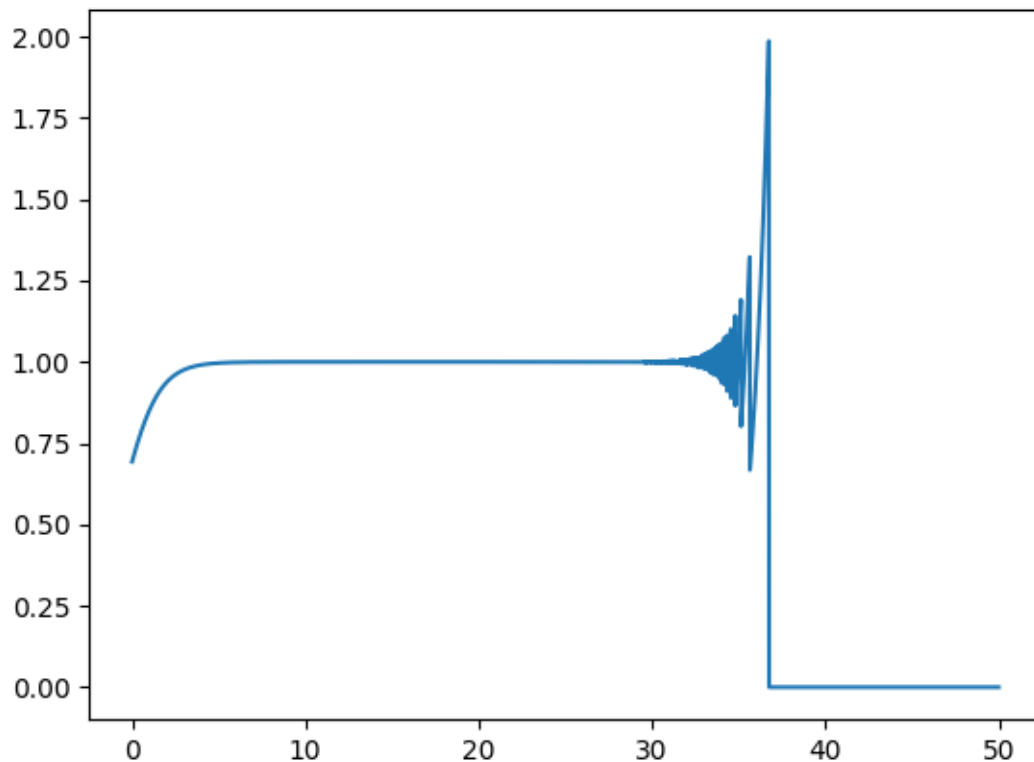
$$\lim_{x \rightarrow \infty} e^x \left(e^{-x} - \frac{e^{-2x}}{2} + \dots \right) = \lim_{x \rightarrow \infty} \left(1 - \frac{e^{-x}}{2} + \dots \right) = 1$$

Granica funkcji wynosi 1.

Wyniki i interpretacja



Rysunek 1: Wykres funkcji $f(x)$ wygenerowany w języku Julia.



Rysunek 2: Wykres funkcji $f(x)$ wygenerowany w języku Python.

Zarówno w Julii jak i w Python'ie początkowo funkcja zbiega do poprawnej granicy - 1, ale po dotarciu do około $x=31$ zaczyna oscylować dookoła wartości 1, następnie po dotarciu do około $x=36$ spada do 0 i tam się już utrzymuje.

Wnioski

Dla dużych wartości x wartość $1 + e^{-x}$ jest ekstremalnie blisko 1 z tego powodu błędy zaokrąglenia mają ogromne znaczenie. Po $x=31$ te błędy zaczynają powodować oscylację, a od momentu gdy $1 + e^{-x}$ jest reprezentowane jako 1 funkcja spada do 0.

Zadanie 3

Krótki opis problemu

Zadanie dotyczy rozwiązania układu równań liniowych $Ax = b$, gdzie dokładne rozwiązanie x jest wektorem złożonym z samych jedynek. Macierz A generowana jest na dwa sposoby: jako macierz Hilberta H_n o rosnącym stopniu n , lub jako macierz losowa R_n o zadanym wskaźniku uwarunkowania c . Układ jest rozwiązywany za pomocą dwóch metod: eliminacji Gaussa (operator ‘\’) oraz poprzez mnożenie przez macierz odwrotną (‘inv(A)*b’). Celem jest zbadanie, jak stopień macierzy n i wskaźnik uwarunkowania c wpływają na błąd względny otrzymanego rozwiązania.

Opis rozwiązania

Eksperyment przeprowadzono dla macierzy o rozmiarach $n \in \{5, 10, 15, 20\}$.

- Dla macierzy Hilberta H_n badamy wpływ wzrostu rozmiaru n na błąd.
- Dla macierzy losowej R_n o kontrolowanym uwarunkowaniu, dla każdego rozmiaru n testowano wskaźniki uwarunkowania $c \in \{10^0, 10^1, 10^3, 10^7, 10^{12}, 10^{16}\}$.

Dla każdego przypadku obliczono błąd względny $\delta = \frac{\|x_{\text{dokładne}} - x_{\text{obliczone}}\|}{\|x_{\text{dokładne}}\|}$.

Wyniki i interpretacja

Poniżej przedstawiono skrócone wyniki dla różnych wartości n i c .

n = 5

```
=====n: 5=====
Hilbert matrix
condition number: 476607.25...
rank: 5
gauss method, delta: 1.68e-12
inverse matrix method, delta: 3.35e-12
```

```
Random matrix, c=1e16
condition number: 5.81e15
rank: 4
gauss method, delta: 0.203
inverse matrix method, delta: 0.195
```

Już dla $n = 5$ macierz Hilberta ma wskaźnik uwarunkowania rzędu 10^5 , ale błędy są jeszcze bardzo małe. Dla macierzy losowej o $c \approx 10^{16}$, błędy są duże, a rząd macierzy spada do 4.

n = 10

```
=====n: 10=====
Hilbert matrix
condition number: 1.60e13
```

```
rank: 10
gauss method, delta: 8.67e-05
inverse matrix method, delta: 0.00025
```

```
Random matrix, c=1e16
condition number: 8.23e15
rank: 9
gauss method, delta: 0.109
inverse matrix method, delta: 0.104
```

Dla $n = 10$ wskaźnik uwarunkowania macierzy Hilberta wzrasta do 10^{13} , a błędy rosną do poziomu 10^{-4} . Macierz losowa o wysokim c znowu traci rząd.

n = 15

```
=====n: 15=====
Hilbert matrix
condition number: 3.67e17
rank: 12
gauss method, delta: 4.69
inverse matrix method, : 7.34
```

```
Random matrix, c=1e16
condition number: 9.27e15
rank: 14
gauss method, delta: 0.583
inverse matrix method, delta: 0.579
```

Dla $n = 15$ błąd względny macierzy Hilberta wynik wzrasta powyżej 1.

n = 20

```
=====n: 20=====
Hilbert matrix
condition number: 1.14e18
rank: 13
gauss method, delta: 108.31
inverse matrix method, delta: 114.34
```

```
Random matrix, c=1e16
condition number: 5.02e15
rank: 19
gauss method, delta: 0.112
inverse matrix method, delta: 0.080
```

Wnioski

Dokładność numerycznego rozwiązania układu równań liniowych jest silnie zależna od wskaźnika uwarunkowania macierzy A .

- Wraz ze wzrostem wskaźnika uwarunkowania, błąd względny rozwiązania rośnie.
- Macierze Hilberta są przykładem macierzy, których uwarunkowanie gwałtownie pogarsza się wraz ze wzrostem ich rozmiaru.
- Metody oparte na eliminacji Gaussa i odwracaniu macierzy dają bardzo podobne błędy i są równie podatne na złe uwarunkowanie problemu. Chociaż na ogół metoda eliminacji Gaussa daje delikatnie lepsze wyniki

Zadanie 4 ("złośliwy wielomian")

Krótki opis problemu

Zadanie polega na zbadaniu uwarunkowania problemu znajdowania pierwiastków wielomianu Wilkinsona, którego pierwiastkami są liczby całkowite od 1 do 20. W części (a) szukamy pierwiastków wielomianu w jego postaci rozwiniętej (z podanymi współczynnikami). W części (b) powtarzamy eksperyment, ale z jednym współczynnikiem ($a_{19} = -210$) minimalnie zaburzonym o wartość -2^{-23} .

Opis rozwiązania

Do znalezienia pierwiastków wielomianu $P(x)$ użyto funkcji ‘roots’ z pakietu ‘Polynomials’ w języku Julia. Dla każdego obliczonego pierwiastka z_k sprawdzono jego dokładność, obliczając:

- $|P(z_k)|$ – wartość wielomianu w postaci rozwiniętej.
- $|p(z_k)|$ – wartość wielomianu w postaci iloczynowej $p(x) = \prod_{i=1}^{20} (x - i)$.
- $|z_k - k|$ – odległość obliczonego pierwiastka od jego prawdziwej wartości.

Wyniki i interpretacja

a) Wielomian oryginalny

Fragment wyników:

```
calculated 1th root z=0.04936236350479326 - 0.001626153012699879im
|P(z)| = 1.8104489942337295e-9
|p(z)| = 2.0329738596513992e18
|z-k| = 0.950639027336251
calculated 2th root z=0.04936236350479326 + 0.001626153012699879im
|P(z)| = 1.8104489942337295e-9
|p(z)| = 2.0329738596513992e18
|z-k| = 1.9506383143179638
calculated 3th root z=0.05273683194109849 - 0.006424840335157523im
|P(z)| = 5.2738229437780115e-9
|p(z)| = 2.0079346511458442e18
|z-k| = 2.947270170915779
calculated 4th root z=0.05273683194109849 + 0.006424840335157523im
|P(z)| = 5.2738229437780115e-9
|p(z)| = 2.0079346511458442e18
|z-k| = 3.947268396814147
...
calculated 16th root z=0.19996988387126097 + 0.0im
|P(z)| = 0.00024684365054636445
|p(z)| = 1.1434141389079118e18
|z-k| = 15.800030116128738
calculated 17th root z=0.2500007072294487 + 0.0im
|P(z)| = 0.0012927904922570654
```

```

|p(z)| = 9.344475883892236e17
|z-k| = 16.74999929277055
calculated 18th root z=0.3333333285199683 + 0.0im
|P(z)| = 0.008840096005399705
|p(z)| = 6.582420831168466e17
|z-k| = 17.66666667148003
calculated 19th root z=0.49999999998784556 + 0.0im
|P(z)| = 0.2954341634259548
|p(z)| = 3.0501459769454624e17
|z-k| = 18.500000000012154
calculated 20th root z=1.0000000000000355 + 0.0im
|P(z)| = 2961.7021218212712
|p(z)| = 4321.702121814633
|z-k| = 18.999999999999964

```

Obliczone pierwiastki są bardzo niedokładne. Wiele z nich ma niezerową część urojoną, a ich odległość od prawdziwych wartości całkowitych jest duża. Wartość $|P(z_k)|$ jest bliska zeru, co sugeruje, że algorytm znalazł "dobre" pierwiastki dla przybliżenia formy rozwinętej, jednak $|p(z_k)|$ jest ogromne, co pokazuje, jak bardzo te wartości odbiegają od prawdziwych pierwiastków.

b) Wielomian z zaburzonym współczynnikiem

Fragment wyników:

```

calculated 1th root z=0.9999999999998357 + 0.0im
|P(z)| = 20259.872313418207
|p(z)| = 19987.872313406842
|z-k| = 1.6431300764452317e-13
calculated 2th root z=2.0000000000550373 + 0.0im
|P(z)| = 346541.4137593836
|p(z)| = 352369.413808796
|z-k| = 5.503730804434781e-11
calculated 3th root z=2.99999999660342 + 0.0im
|P(z)| = 2.2580597001197007e6
|p(z)| = 2.416241558251844e6
|z-k| = 3.3965799062229962e-9
calculated 4th root z=4.000000089724362 + 0.0im
|P(z)| = 1.0542631790395478e7
|p(z)| = 1.126370230029202e7
|z-k| = 8.972436216225788e-8
...
calculated 16th root z=16.73074487979267 - 2.812624896721978im
|P(z)| = 4.850110893921027e11
|p(z)| = 2.742106076928478e16
|z-k| = 2.9060018735375106
calculated 17th root z=16.73074487979267 + 2.812624896721978im
|P(z)| = 4.850110893921027e11

```

```

|p(z)| = 2.742106076928478e16
|z-k| = 2.825483521349608
calculated 18th root z=19.5024423688181 - 1.940331978642903im
|P(z)| = 4.557199223869993e12
|p(z)| = 4.25248587652037e17
|z-k| = 2.4540214463129764
calculated 19th root z=19.5024423688181 + 1.940331978642903im
|P(z)| = 4.557199223869993e12
|p(z)| = 4.25248587652037e17
|z-k| = 2.0043294443099486
calculated 20th root z=20.84691021519479 + 0.0im
|P(z)| = 8.756386551865696e12
|p(z)| = 1.3743743559997601e18
|z-k| = 0.8469102151947894

```

Minimalne zaburzenie jednego współczynnika (zmiana o ok. 10^{-7}) spowodowało katastrofalne zmiany w pierwiastkach. Pierwiastki, które były bliskie 1-9, pozostały stosunkowo dokładne. Jednak pierwiastki "środkowe"(10-18) uległy drastycznemu przesunięciu, a większość z nich stała się liczbami zespolonymi o dużych częściach urojonych. Błąd $|z_k - k|$ jest znaczny.

Wnioski

Problem znajdowania pierwiastków wielomianu na podstawie jego współczynników jest **źle uwarunkowany**. Wielomian Wilkinsona to obrazuje. Niewielka zmiana w jednym ze współczynników prowadzi do ogromnych zmian w położeniu pierwiastków. Pokazuje to, że reprezentacja wielomianu poprzez jego współczynniki jest numerycznie niestabilna do tego typu obliczeń, a o wiele lepsza jest forma iloczynowa.

Zadanie 5

Krótki opis problemu

Zadanie dotyczy analizy równania rekurencyjnego znanego jako model logistyczny: $p_{n+1} = p_n + rp_n(1 - p_n)$, dla $p_0 = 0.01$ i $r = 3$. Należy porównać wyniki 40 iteracji tego równania w dwóch scenariuszach:

1. Standardowa symulacja przez 40 kroków.
2. Symulacja przerywana po 10 krokach, wynik obcięty do 3 miejsc po przecinku (zaokrąglony w dół), a następnie kontynuowana przez kolejne 30 kroków.

Eksperymenty przeprowadzono w arytmetyce ‘Float32’ i ‘Float64’.

Opis rozwiązania

Zaimplementowano funkcję, która iteruje równanie logistyczne. Następnie uruchomiono ją dla obu scenariuszy i obu typów arytmetyki, porównując końcowe wartości p_{40} .

Wyniki i interpretacja

```
===== Type: Float32 =====  
p calculated in a standard way: 0.089340806  
p calculated after cutoff (10-th round): 0.7229306  
p calculated after cutoff (40-th round): 0.7715323  
  
===== Type: Float64 =====  
p calculated in a standard way: 0.011601082861106218  
p calculated after cutoff (10-th round): 0.7229143011796237  
p calculated after cutoff (40-th round): 0.7305550513325518
```

Wyniki pokazują, że obcięcie precyzji w 10. kroku symulacji prowadzi do całkowicie odmiennych wyników końcowych w obu typach arytmetyki. W ‘Float32’ wynik standardowy to ≈ 0.089 , a po obcięciu ≈ 0.771 . W ‘Float64’ różnica jest równie duża: ≈ 0.011 w porównaniu do ≈ 0.730 . Dodatkowo różnice między tymi wynikami w tych dwóch typach arytmetyki są znaczne.

Wnioski

Obcięcie wyniku po 10 iteracjach jest równoważne z wprowadzeniem niewielkiego błędu, który jest następnie wzmacniany w kolejnych krokach, prowadząc do zupełnie innych rezultatów. Długoterminowe przewidywanie systemów chaotycznych jest niestabilne numerycznie.

Zadanie 6

Krótki opis problemu

Zadanie polega na obserwacji zachowania ciągów generowanych przez równanie rekurencyjne $x_{n+1} = x_n^2 + c$ dla 40 iteracji. Należy przeprowadzić eksperymenty dla siedmiu różnych par danych początkowych (c, x_0) .

Opis rozwiązania

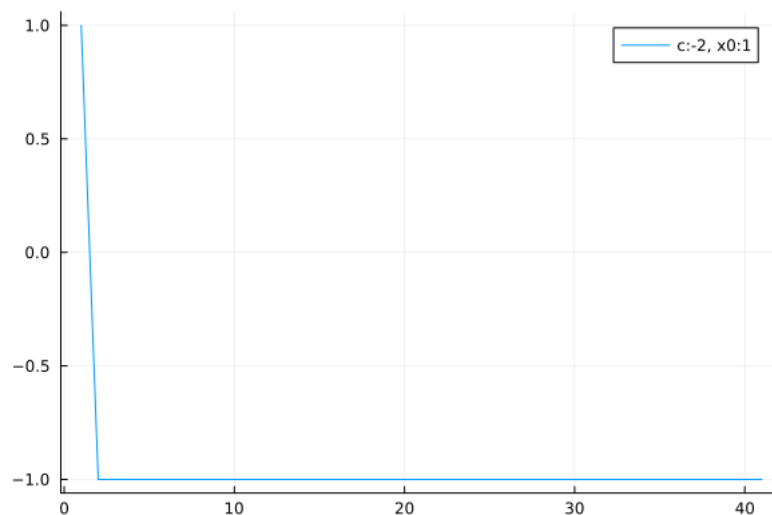
Dla każdej z siedmiu par (c, x_0) wygenerowano 40 kolejnych wyrazów ciągu, zapisując ich wartości. Wyniki zostały również zwizualizowane na wykresach. Obliczenia przeprowadzono w arytmetyce 'Float64'.

Wyniki i interpretacja

Poniżej opisano zachowanie ciągów dla poszczególnych danych wejściowych.

1. $c = -2, x_0 = 1$ Ciąg po pierwszym kroku ($x_1 = 1^2 - 2 = -1$) stabilizuje się na stałej wartości -1.

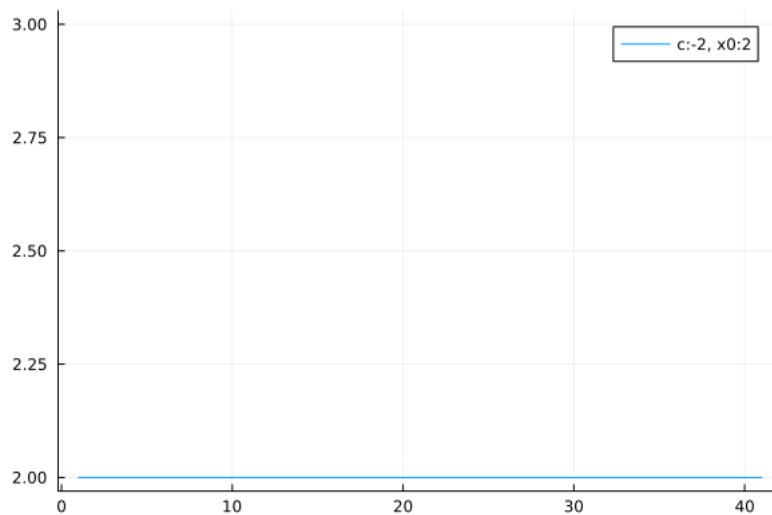
```
1th element: 1
2th element: -1
3th element: -1
...
41th element: -1
```



Rysunek 3: Wykres dla $c = -2, x_0 = 1$.

2. $c = -2, x_0 = 2$ Ciąg od samego początku jest w punkcie stałym ($x_1 = 2^2 - 2 = 2$).

```
1th element: 2
2th element: 2
...
41th element: 2
```



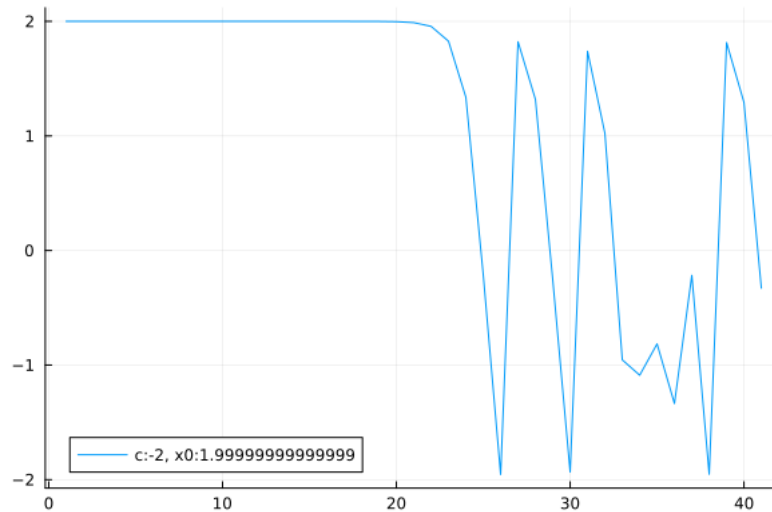
Rysunek 4: Wykres dla $c = -2, x_0 = 2$.

3. $c = -2, x_0 = 1.999999999999999$ Niewielkie zaburzenie punktu stałego $x_0 = 2$ prowadzi do zmiany zachowania. Początkowo ciąg maleje, zbliżając się do wartości -2, a następnie zaczyna chaotycznie oscylować w przedziale $[-2, 2]$.

```

1th element: 1.999999999999999
2th element: 1.999999999999996
3th element: 1.99999999999998401
4th element: 1.99999999999993605
5th element: 1.9999999999997442
6th element: 1.99999999999897682
7th element: 1.99999999999590727
8th element: 1.999999999836291
9th element: 1.9999999993451638
...
24th element: 1.3371201625639997
25th element: -0.21210967086482313
26th element: -1.9550094875256163
27th element: 1.822062096315173
28th element: 1.319910282828443
29th element: -0.2578368452837396
...
39th element: 1.8145742550678174
40th element: 1.2926797271549244
41th element: -0.3289791230026702

```



Rysunek 5: Wykres dla $c = -2, x_0 = 1.999\dots$

4. $c = -1, x_0 = 1$ oraz 5. $c = -1, x_0 = -1$ W obu przypadkach ciąg natychmiast wpada w cykl o okresie 2: $(0, -1, 0, -1, \dots)$.

SIMULATION: $c:-1, x0: 1$

1th element: 1

2th element: 0

3th element: -1

4th element: 0

5th element: -1

6th element: 0

7th element: -1

8th element: 0

...

SIMULATION: $c:-1, x0: -1$

1th element: -1

2th element: 0

3th element: -1

4th element: 0

5th element: -1

6th element: 0

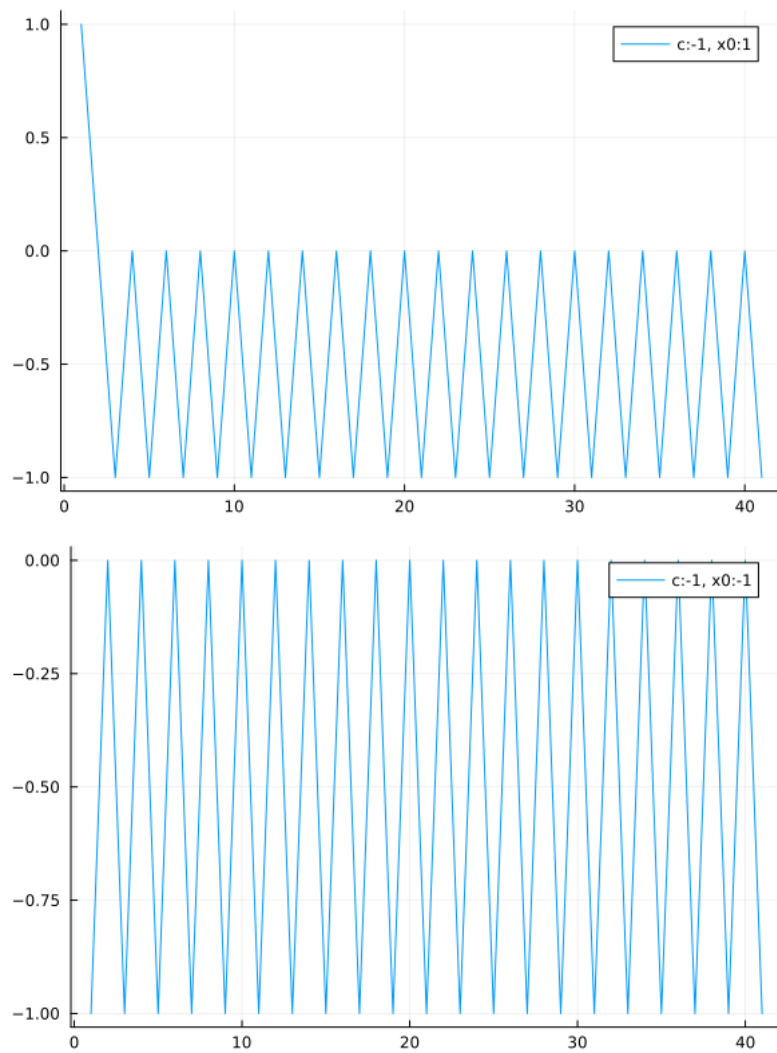
7th element: -1

8th element: 0

9th element: -1

10th element: 0

11th element: -1



Rysunek 6: Wykres dla $c = -1$, $x_0 = 1$ lub $x_0 = -1$.

6. $c = -1$, $x_0 = 0.75$ oraz 7. $c = -1$, $x_0 = 0.25$ W obu przypadkach ciąg jest zbieżny do cyklu $(0, -1)$. Po kilkunastu iteracjach wartości stają się praktycznie nierozróżnialne od 0 i -1.

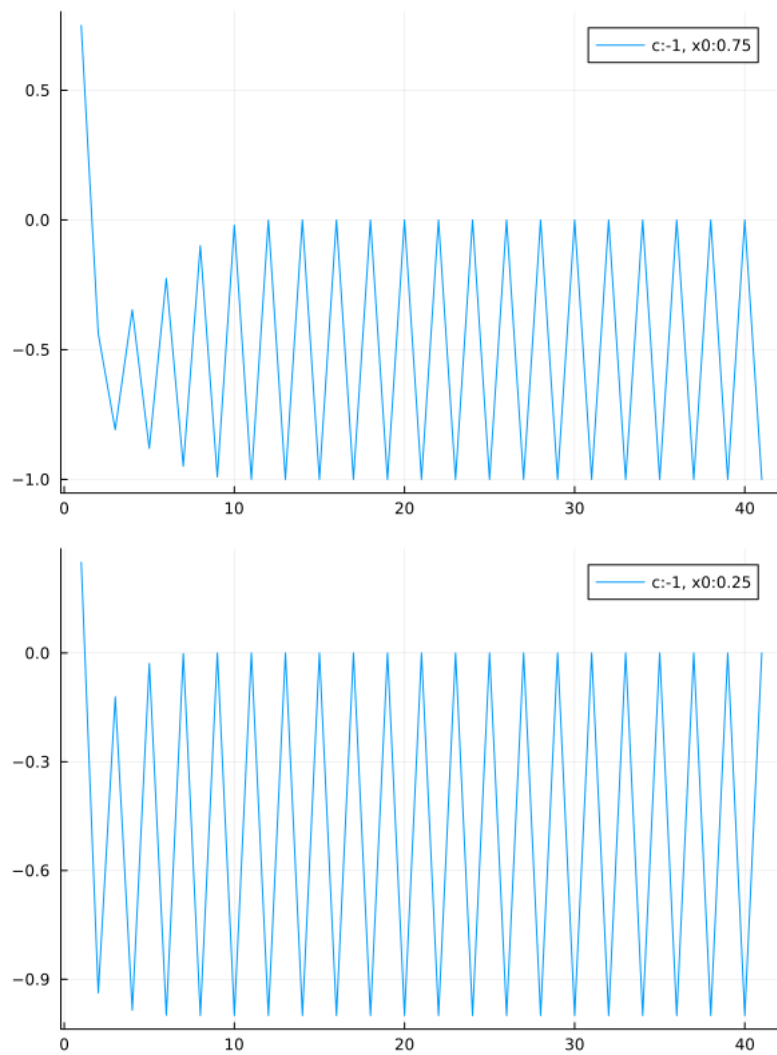
```
SIMULATION: c:-1, x0: 0.75
1th element: 0.75
2th element: -0.4375
3th element: -0.80859375
4th element: -0.3461761474609375
5th element: -0.8801620749291033
...
14th element: -1.1536182557003727e-6
15th element: -0.9999999999986692
16th element: -2.6616486792363503e-12
17th element: -1.0
18th element: 0.0
19th element: -1.0
20th element: 0.0
```



```

...
SIMULATION: c:-1, x0: 0.25
1th element: 0.25
2th element: -0.9375
3th element: -0.12109375
4th element: -0.9853363037109375
5th element: -0.029112368589267135
...
10th element: -0.9999999999670343
11th element: -6.593148249578462e-11
12th element: -1.0
13th element: 0.0
14th element: -1.0
15th element: 0.0
...

```



Rysunek 7: Wykres dla $c = -1$, $x_0 = 0.75$ lub $x_0 = 0.25$.

Wnioski

Eksperymenty pokazały różnorodne zachowania systemu w zależności od parametrów:

- **Punkty stałe** (np. $c = -2, x_0 = 2$).
- **Oscylacje** - ciągi oscylujące między dwiema wartościami (np. dla $c = -1$ $x_0 = 1$).
- **Zbieganie do oscylacji** - ciągi które z większą ilością kroków zaczynają oscylować między dwiema wartościami (np. dla $c = -1$ i $x_0 = 0.25$)
- **Wrażliwość na warunki początkowe** – niewielka zmiana x_0 (z 2 na 1.999...) dla $c = -2$.

Obrazuje to złożoność wynikającą z nieliniowych układów.